A common thread throughout this discussion of MVC is the separation of concerns MVC brings, and it's no different when it comes to your views. It would be easy enough to generate the HTML directly in your application model or in your controller actions, but instead you delegate that responsibility to a single component: the view.

But even more than that, you separate the *data* required to build the view from the *process* of building it by using properties on the `PageModel`. These properties should contain all the dynamic data the view needs to generate the final output.

TIP Views shouldn't call methods on the `PageModel`. The view should generally only be accessing data that has already been collected and exposed as properties.

Razor Page handlers indicate that the Razor view should be rendered by returning a `PageResult` (or by returning `void`), as you saw in chapter 15. The Razor Pages infrastructure executes the Razor view associated with a given Razor Page to generate the final response. The use of C# in the Razor template means you can dynamically generate the final HTML sent to the browser. This allows you to, for example, display the name of the current user in the page, hide links the current user doesn't have access to, or render a button for every item in a list.

Imagine your boss asks you to add a page to your application that displays a list of the application's users. You should also be able to view a user from the page or create a new one, as shown in figure 17.2.